

TRACEY (traceyDB) — Architecture & Maintenance Manual

This document explain the project's architecture, the main views, the admin panel (features.html), and the frontend structure, so that anyone picking up the project can maintain it without having to reread the whole codebase from scratch.

Running the dev server

```
/home/cpulidoq/anaconda3/envs/django/bin/python /mnt/c/Users/cpulidoq/Documents/traceyDB/manage.py runserver
localhost:8000
```

Python interpreter

WSL path: /home/cpulidoq/anaconda3/envs/django/bin/python

Table of contents

1. [General architecture](#)
 2. [Reference of the main views \(query-*\)](#)
 3. [features.html — admin panel](#)
 4. [Frontend: templates, JS, includes and menus](#)
 5. [Maintenance commands \(manage.py\)](#)
 6. [Notes and gotchas for the next developer](#)
 7. [Note on this document and exporting to PDF](#)
-

1. General architecture

1.1 Repository layout

core/	Django project (settings.py, urls.py, wsgi.py, asgi.py)
apps/	only app registered in INSTALLED_APPS ("apps.config.AppsConfig")
├─ home/	the "real" app: views, forms, models, plots, utils
├─ authentication/	login/register/logout
├─ management/commands/	custom manage.py commands (see section 5)
├─ templates/	layouts, includes, menus, home/ pages (see section 4)
├─ static/	CSS/SCSS, JS, images (see section 4)
utils/	bioinformatics scripts (HMM, BLAST, NCBI taxonomy, phylogenetic trees)
	NOT a Django app, these are scripts invoked by the management commands
docs/	this manual

Prior diagrams (not duplicated here, useful as visual reference): traceyStructure.pdf/pptx, tracey_all_models.pdf, tracey_main_models.pdf, tracey_other_models.pdf, DomaingroupsStructure.pdf/pptx, tree.pdf (all at the repo root).

1.2 The INSTALLED_APPS quirk

In core/settings.py, INSTALLED_APPS only registers 'apps.config.AppsConfig' — it does **not** register apps.home or apps.authentication as separate apps. Since Django's app registry assigns models by module-path prefix matching, models defined in apps/home/models.py end up under the apps app-label. That's why apps/admin.py registers the admin by iterating apps.get_app_config('apps') instead of apps.get_app_config('home'). The HomeConfig class in apps/home/apps.py exists but, since it's not in INSTALLED_APPS, is effectively dead code.

1.3 Database

The backend is MySQL (core/settings.py, ENGINE: django.db.backends.mysql), pointing at a **pre-existing** schema. All models in apps/home/models.py have managed = False. This means:

- Schema changes are **not** done with makemigrations/migrate. The MySQL database has to be modified directly (or via the .sql dumps in utils/), and the change then reflected by hand in models.py.
- apps/migrations/ only has 0001_initial.py — there is no real migration history to maintain.

1.4 Routing (core/urls.py → apps/home/urls.py)

core/urls.py includes, in order: admin/, apps.authentication.urls (login/register/logout), captcha/, robots.txt, sitemap.xml, and **last** apps.home.urls. A comment in the file itself warns that apps.home.urls must stay last, because it ends with a catch-all:

```
re_path(r'^.*\.*$', views.pages, name='pages')
```

The pages() view (in apps/home/views_query.py) takes the last URL segment and loads home/<segment>.html directly as a template — that's how standalone pages like /contact.html work without an explicit route. updates.html is

gated behind login via the `_PAGES_REQUIRES_LOGIN` set.

1.5 The “grouped views” pattern

Views are split across several modules inside `apps/home/`, grouped by functional family, and `views.py` acts as a **single aggregator**: it does explicit named imports from each module and re-exports them. `urls.py` only does `from apps.home import views` and calls `views.<Name>` — it never imports a sub-module directly.

Module	View family
<code>apps/home/views_query.py</code>	<code>query-sequences</code> , <code>query-sequences-results</code> , FASTA/multialignment export, 3D viewer, <code>query-sequence-details</code> , ajax endpoints for autocomplete and cascading dropdowns
<code>apps/home/views_motifs.py</code>	<code>query-motifs</code> , <code>query-motifs-results</code> , and the <code>motifScan</code> helper (shared HMM-scanning engine)
<code>apps/home/views_verify.py</code>	<code>query-insert</code> , <code>query-verify</code> (curation menu and view), <code>query-verify-blast</code>
<code>apps/home/views_trees.py</code>	Phylogenetic tree view (trees)
<code>apps/home/views_admin.py</code>	<code>features.html</code> and all its actions (see section 3)

If a new view is added, it must be added both in its corresponding `views_*.py` module and in the explicit import list in `apps/home/views.py` so it’s exposed to `urls.py`.

1.6 The segment convention (menu highlighting)

Each view manually computes a segment context variable (typically `request.path.split('/')[1]` or a variant) and passes it to the template. `includes/sidebar.html` uses that variable in conditionals such as `{% if 'query' in segment %}` to highlight the active menu item.

There is no central context processor for this — it’s a pattern copy-pasted view by view. If a new view is added and segment is forgotten, the sidebar highlighting simply won’t work for that page (a silent failure, no error).

1.7 Key models

All in `apps/home/models.py`, `managed = False` (see the model PDFs at the repo root for the full schema):

Model	Purpose
Sequences	Central entity: a protein/gene sequence (annotation, status, taxonomy, associated gene)
Motifs	Verified/active HMM motif hits on a sequence
Verifymotifs	Mirror of Motifs for hits pending verification (the <code>query-verify</code> workflow)
Domaingroups	Functional/structural domain groups, with a parent hierarchy (<code>domaingroupparent_id</code> , can have several parents separated by ;)
Domains	Top-level protein domain (SNARE, Habc, Longin, LGL, ...)
Taxonomies	Node in TRACEY’s own taxonomy tree (scientific name, rank, parent)
Methods	How a motif/verifymotif was generated (<code>type = 'hmm' or 'manual'</code>)

2. Reference of the main views (query-*)

QuerySequences / QuerySequencesResults

File: `apps/home/views_query.py`. URLs: `query-sequences`, `query-sequences-results`.

- `QuerySequences` renders the search form (`home/query-sequences.html`). Key context: `domainsList`, `proteinLayoutsList` (from `get_menu(request)`, distinguishes staff vs public menu), `shortnames` (cached 24h), `taxonomy_ranks`, `form` (`FamilyForm`), `is_staff`, `error` (session message if the previous search returned 0 results).
- `QuerySequencesResults` runs the actual search by delegating to `get_sequences()` (`apps/home/utls.py`), which combines non-motif filters (`aliases`, `status`, `species`, `taxonomy subtree` via BFS) with motif filters (`domain/domaingroup/proteinlayout`), and splits results into `verified` (`Motifs`) vs `unverified` (`Verifymotifs`). Hard cap of 4000 rows. Key context: `sequences`, `speciesname`, `motifnames`, `hmmModels` (list of available `.hmm` files for generating an MSA).

QuerySequencesFastaFormat

URL: `query-sequences-fasta` (POST).

Export endpoint with 4 modes depending on which POST field is present: TSV table download, plain per-sequence FASTA, per-motif FASTA (extracted from the XML in asciioutput), or HMM multialignment (via `pyhmmer.hmmalign`) to view/download.

QuerySequences3dViewer

URL: `query-sequences-3dViewer/<sequence_id>`.

Renders the 3D viewer (3Dmol.js) for a sequence. Key context: `sequence`, `fetch3d` (AlphaFold URL), `motif_coords` (real coordinates of each domaingroup, recomputed against the full sequence). Uses `user_can_access_sequence()` (`apps/home/utlils.py`) to verify that all of the sequence's domaingroups are within the menu allowed for the caller (staff vs public); raises `Http404` otherwise.

QuerySequencesDetails (query-sequence-details)

URL: `query-sequences/<sequence_id>/details/`.

Sequence detail page. Key context: `sequence`, `speciesname`, `wiki_image`, `pdb/pdb_name` (parsed out of `foreignannotation`), `fetch3d`, `layout` (SVG plot via `build_domain_plot()`), and `motifs` — a dict per active Motifs entry with its domaingroup-parent chain, fields parsed out of the asciioutput XML (consensus, similarity, e-value), and an individual plot per motif. Same access check as the 3D viewer.

DetailsSequencesFastaFormat

URL: `details/fastaFormat/<sequence_id>/`. Simple FASTA download of a sequence, with the same access control.

QueryMotifsView / QueryMotifsResultsView

File: `apps/home/views_motifs.py`. URLs: `query-motifs`, `query-motifs-results`.

Form to scan a hand-pasted sequence against TRACEY's HMMs (no need for the sequence to already exist in the database). `QueryMotifsView` validates the input (non-empty, ≤ 2000 aa) and redirects to `QueryMotifsResultsView`, which calls the `motifScan()` helper.

`motifScan()` (same file) is the HMM-scanning engine also reused by `QueryInsertView` and `QueryVerifyView`: it validates the input isn't FASTA-formatted, runs `pyhmmer` against the relevant subset of `.hmm` files (or the full `MOTIFS.hmmDb` if `proteinlayout == "ALL"`), filters by `evalcutoff`, resolves domaingroup/domain names and real alignment coordinates, and returns `hits_d` (a hits dict sorted by ascending p-value).

QueryInsertView (query-insert)

File: `apps/home/views_verify.py`. Staff-only (@login_required + @staff_login_required).

Form to insert a new sequence (`InsertSequence ModelForm`). On save, it creates the Sequences row, immediately runs `motifScan(..., ["ALL"])`, persists the hits as `Verifymotifs` via `saveVerifyMotifs()`, and redirects to `query-verify` for that sequence. If saving or scanning fails, it rolls back by deleting the newly created Genes row.

QueryVerifyMenuView / QueryVerifyView (query-verify)

File: `apps/home/views_verify.py`. Staff-only.

- `QueryVerifyMenuView` (`query-verify`, no id) is the listing/menu page for sequences pending curation; the actual table is loaded via ajax through `load_queryverifysequences` (`apps/home/views_query.py`, partial template `home/query-verify-update-sequences.html`).
- `QueryVerifyView` (`query-verify/<sequence_id>`) is the curation workbench view, the most complex view in the project. From the same form you can: verify/promote a `Verifymotifs` to `Motifs` (or the reverse, "unverify"), delete motifs or the whole sequence, re-scan (`motifScan` + `saveVerifyMotifs`), or add a manual motif (`add_manual_motif`, creates a `Methods` row with `type='manual'`). Each action appends an audit line to the sequence's changelog field.

QueryVerifyBlastView (query-verify-blast)

URL: `query-verify/traceyBLAST/<db>/<query_id>`.

Compares a sequence/motif/verifymotif against one of TRACEY's local BLAST databases (or redirects to public NCBI BLAST if `db` indicates so), using the local `blastp` binary at `utils/ncbi-blast-2.13.0+/bin/blastp`, and renders the result as a colored alignment viewer.

Supporting ajax endpoints

A set of small GET views in `views_query.py` (`load_taxonomy_rank`, `load_species`, `load_domains`, `load_domaingroups_rank1/2`, `load_sequenceshortnames`, `load_taxonomy_by_shortcode`, `suggest_aliases`, `updateSequenceStatus`) and in `views_verify.py` (`suggestNames`, `autocompleteModel`) feed the cascading dropdowns and autocomplete of the search/insert/verify forms. They're not detailed one by one here — they all follow the same pattern: receive a parameter from the level above and return partial HTML or JSON with the options for the next level.

3. features.html — admin panel

Template: `apps/templates/home/features.html`. View: `features()` in `apps/home/views_admin.py`. Staff-only.

This is TRACEY's data-maintenance panel: six cards, each with its own form + ajax call + inline script at the bottom of the file.

Feature	What it does	Ajax endpoint	Script/command behind it
Update Taxonomies	Refreshes the taxonomy table against current NCBI data; creates new taxa if needed. Takes ~10 min.	<code>ajax_update_taxonomy</code> → <code>views.update_taxonomy</code>	<code>utils/ncbi_taxonomy/TaxonomyUpdater.py</code> , same engine as <code>manage.py UpdateTraceyTaxonomies</code>
Update TRACEY Sequences	Re-syncs "live" sequences that have an NCBI ID against current data (sequences without an NCBI ID are left untouched). Filterable by species/family. Can take hours.	<code>ajax_update_sequences</code> → <code>views.update_sequences</code>	<code>utils/traceySequenceUpdater/traceySequencesUpdater.py</code> , same engine as <code>manage.py UpdateTraceySequences</code>
Re-scan Motifs	Re-runs the HMM scan over existing sequences to refresh Motifs/Verifymotifs. Filters: species, family/domain/specific HMM, e-value threshold, "only active", dry-run mode (opens the result in a new tab without persisting).	<code>ajax_rescan_motifs</code> → <code>views.rescan_motifs</code>	<code>utils/motifPredictor/reScanMotifs.py</code> , same engine as <code>manage.py ReScanMotifs</code>
Upload new sequences	Uploads a FASTA of new sequences; TRACEY analyzes them against the HMMs and auto-assigns the best-hit motif.	<code>ajax_upload_sequences</code> → <code>views.upload_sequences</code>	Required FASTA header format documented in <code>home/help_doc_upload_sequences.html</code>
HMM Models (download)	Downloads TRACEY's HMM profiles (one family or the full database) for external use with <code>hmmsearch/hmmscan</code> .	<code>download_hmm_zip</code> (direct navigation, not ajax)	Catalogs <code>.hmm</code> files in <code>utils/hmmModels/</code> via <code>get_hmm_catalog()</code>
Build Phylogenetic Tree	Rebuilds the tree of life for all taxonomies currently in TRACEY, following current NCBI phylogeny. Provides download links for the tree in Newick and PDF form.	<code>ajax_update_tree</code> → <code>views.update_tree</code>	<code>utils/ncbi_taxonomy/TreeUpdater.py</code> , same engine as <code>manage.py UpdateTraceyTree</code>

Common pattern across all six: a CSRF token in each form, a spinner + status message, and a shared flag (`reloadOnAjaxStop`) that reloads the page once any of these long ajax calls finishes, to refresh the "last updated" timestamps.

4. Frontend: templates, JS, includes and menus

4.1 Template tree (`apps/templates/`)

<code>layouts/</code>	
<code>base.html</code>	main layout (sidebar + content + footer)
<code>base-fullscreen.html</code>	sidebar-less variant (appears UNUSED currently, see section 6)
<code>sequenceForm.html</code>	large insert/verify-sequence form (an include, not a base)
<code>includes/</code>	
<code>sidebar.html</code>	navigation menu (hardcoded, see 4.3)
<code>footer.html</code>	page footer (partner logos)
<code>scripts.html</code>	common JS bundle (Popper, Bootstrap, Chartist, SweetAlert2, moment.js, volt.js, ...)

settings-box.html	Volt theme widget (commented out in base.html, unused)
accounts/	login.html, register.html
home/	~30 page templates + partials/_motif_card.html
menus/	NOT HTML templates – Python modules (see 4.3)

layouts/base.html defines the blocks title, og_title, og_description, stylesheets, content, javascripts. Most templates in home/ and accounts/ extend it directly (a single level of inheritance, no deeper chains).

Some templates in home/ are **not full pages**: they’re HTML fragments returned directly by ajax views (e.g. query-sequences-fasta.html, query-verify-update-sequences.html, dropdown option lists). Others are standalone HTML documents opened in a new tab (query-sequences-3dViewer.html, query-verify-blast.html, each with its own `<!DOCTYPE html>`).

home/partial/_motif_card.html is reused twice in layouts/sequenceForm.html: once for verifymotifs (with a Verify/Delete option) and once for motifs (with an Unverify/Delete option).

4.2 Main JS files (apps/static/assets/js/)

File	Role
volt.js	JS for the Volt theme (Bootstrap 5): SweetAlert2, settings panel, general theme chrome
sequenceForm-dropdowns.js	Cascading dropdowns proteinlayout → domain → domaingroup → domainsubgroup on the insert/verify form (and its “manual” variant)
sequenceForm-utils.js	suggestNames() (ajax to ajax_suggestNames) and searchOpen() (taxonomy autocomplete via /search.json)
ngl-viewer.js	3D viewer based on NGL.js , used on the insert/verify sequence form (colors by domain, highlights residues)
django-htmx.js	Debug helper for django_htmx: when DEBUG=True, replaces HTMX error responses with Django’s debug error page
jquery-4.0.0.min.js	jQuery, loaded per-page (not globally from base.html) wherever AJAX/DOM manipulation is needed

Important: the standalone detail/3D-viewer pages (query-sequences-3dViewer.html, query-sequences-details.html) instead use **3Dmol.js via CDN** (<https://3Dmol.org/build/3Dmol-min.js>). In other words, TRACEY has **two different 3D viewers** depending on the page: NGL.js (local) in the insert/verify workflow, and 3Dmol.js (CDN) on the public query pages. If you touch 3D-visualization functionality, check both.

4.3 How TRACEY builds its menus

There are two distinct concepts both loosely called “menu” — worth not confusing:

- Navigation menu (sidebar)** — includes/sidebar.html. This is **hardcoded** HTML, not generated from the database or from a Python data structure. Active-item highlighting is done with Django conditionals (`{% if 'query' in segment %}`) over the segment variable each view computes by hand (see 1.6). The only real context processor in play is apps/context_processors.py:cfg_assets_root, which just injects ASSETS_ROOT for static-asset paths.
- “Menu” of protein families** — apps/templates/menus/query_sequences.py and query_sequences_full.py. Despite living under templates/ and having a .py extension, **these are not Django templates**: they’re Python modules holding a nested dict (menu = {...}) describing the protein-family → domain → domaingroup hierarchy (SNARE, Ras, AAA, ...), plus helper functions (get_keys_recursively(), get_dict()). Views import them to populate the cascading dropdowns on Query/Insert/Verify/Features. Unrelated to UI navigation.

4.4 CSS / SCSS

Base theme “Volt” (Bootstrap 5, AppSeed/Themesberg), compiled from SCSS (apps/static/assets/scss/) to CSS (apps/static/assets/css/volt.css). Project-specific customization in scss/custom/_variables.scss and css/sequenceForm.css (styles for the insert/verify form). The CSS specific to features.html (loaders, .feature-card grid) is inline in the template itself rather than in a separate file.

5. Maintenance commands (manage.py)

Defined in apps/management/commands/. Each one has an equivalent feature in features.html (section 3) that triggers the same logic from the UI:

Command	What it does
UpdateTraceySequences	Refreshes “live” sequences that have an NCBI ID. Flags: --continue, --force, --onlyActive, --species, --traceyIds, --domain
UpdateTraceyTaxonomies	Refreshes taxonomies from NCBI. Flag --taxa (default superkingdom)

UpdateTraceyTree	Rebuilds the phylogenetic tree
ReScanMotifs	Re-scans HMMs over existing sequences. Requires --hmm or --family
plotTaxonomy	Diagnostic command: draws the taxonomy tree/network for a given scientific name (not part of the data pipeline)

6. Notes and gotchas for the next developer

- **Database schema changes are NOT done with makemigrations/migrate** — all models are managed = False over a pre-existing MySQL database (see 1.3).
 - HomeConfig (apps/home/apps.py) is not in INSTALLED_APPS — appears to be dead code.
 - layouts/base-fullscreen.html is not extended by any current template — verify whether it's still needed before touching or removing it.
 - jquery-3.3.1.min.js and the paginatedTable.js script don't appear to be actively referenced (the only <script> using paginatedTable.js is commented out in query-sequences-results.html) — confirm before assuming they're in use.
 - The PollsChoice/PollsQuestion models in models.py are leftovers from the Django/AppSeed tutorial boilerplate and don't appear related to TRACEY.
 - There are two full MySQL dumps inside utils/ (tracey_20180606_0400.dump2, tracey_20251208.backup.sql, ~1.6-1.7 GB each) checked into the repo — worth removing them from version control (or using Git LFS) unless there's a deliberate reason to keep them there.
 - TRACEY has **two different 3D viewers** (NGL.js vs 3Dmol.js via CDN) depending on the page — see 4.2.
 - Menu highlighting depends on every new view manually defining segment — there's no automatic mechanism for this (see 1.6).
-

7. Note on this document and exporting to PDF

This manual is deliberately kept in Markdown: it's versioned alongside the code, editable without special tooling, and renders directly on GitHub/GitLab/VSCode. If a PDF version is ever needed, it can be generated with:

```
pandoc docs/MANUAL_en.md -o manual.pdf
```

This environment's WSL already has pandoc installed, but **not** a PDF rendering engine (pdflatex/xelatex, wkhtmltopdf, or weasyprint) — one of those would need to be installed before the command above works.